

SENAI - CRTI

Atividade Prática

Caça-Falhas: Auditoria de Tratamento de Erros

Interação com APIs — Sprint 3 — Aula 07 (Tratamento de Falhas e Mensagens)

1. Objetivo da atividade

Nesta atividade você vai auditar a Minha Loja — sua própria aplicação de e-commerce em Angular 22, construída ao longo das aulas anteriores — em busca de falhas ou lacunas no tratamento de erros. Diferente do ViaCep e do JSONPlaceholder (apps mais simples, de uma tela só), a Minha Loja tem catálogo de produtos consumido de uma API real (fakestoreapi.com), carrinho e autenticação persistidos em localStorage, rotas protegidas por guards (login e perfil de administrador) e um interceptor HTTP central que trata erros de rede para toda a aplicação. Isso muda o tipo de falha que vale a pena caçar.

Você vai atuar como revisor de código e testador de QA do seu próprio projeto: ler o código-fonte, testar cenários de falha manualmente no navegador e registrar evidências de tudo o que verificar — seja um problema real, seja a confirmação de que algo já está bem tratado.

2. Por que fazer isso, mesmo sabendo que as falhas são mínimas?

Aviso importante: a Minha Loja já foi revisada e o tratamento de erros nela está, no geral, sólido — guards bem definidos, interceptor HTTP centralizado, mensagens amigáveis no catálogo. É esperado que você não encontre problemas graves, e está tudo bem. O valor real desta atividade está no processo: testar casos-limite, ler código com espírito crítico e documentar evidências, mesmo quando tudo parece funcionar — exatamente como um revisor de código faz no mercado de trabalho, revisão após revisão, na maioria das vezes sem achar nada grave.

Um relatório honesto dizendo “testei X, Y e Z e não encontrei falha, e aqui está a evidência” tem tanto valor quanto um relatório que encontra um problema real. Não é necessário inventar um bug para ter uma boa entrega.

3. Antes de começar

- Abra o projeto Minha Loja no editor de código e suba com "npm start" (ou "ng serve").
- Abra o DevTools do navegador (F12) e deixe as abas Console e Network visíveis durante todos os testes.
- Leia (ou releia) estes arquivos antes de começar: core/interceptors/http.interceptor.ts, core/auth.guard.ts, core/admin.guard.ts, core/services/produtos.service.ts, core/services/carrinho.service.ts, core/services/auth.service.ts e features/produtos/lista-produtos/lista-produtos.ts.
- Identifique as duas contas de teste: e-mail admin@email.com vira administrador; qualquer outro e-mail válido vira usuário comum. A senha só precisa ter 3+ caracteres (não existe validação real de senha).

4. Metodologia de investigação

A Minha Loja tem quatro frentes de falha diferentes das aulas anteriores. Para cada uma, percorra os pontos abaixo:

4.1 Falhas de API (catálogo de produtos)

- O callback "error" do subscribe em lista-produtos.ts está tipado como `HttpErrorResponse`, ou está genérico?
- Existe estado de loading, de erro (com botão de tentar novamente) e de sucesso? E se a API responder uma lista vazia — existe uma mensagem de “nenhum produto encontrado”, ou a tela fica só com “Total de produtos: 0” sem explicação?
- O tratamento de erro do `HttpClient` está duplicado no componente, no interceptor, ou nos dois? Isso é redundância ou é uma divisão de responsabilidades proposital (um trata globalmente, outro trata na tela)?

4.2 Interceptor HTTP central

- Abra `core/interceptors/http.interceptor.ts`. Ele trata 401, 403 e 500 de formas diferentes — confirme cada um provocando o cenário real (veja a seção 5) e observando para onde a navegação vai.
- O interceptor usa `console.log/console.warn/console.error` em toda requisição. Isso é aceitável em um projeto de estudo? O que você mudaria antes de considerar este código pronto para produção?

4.3 Autenticação e autorização (guards)

- O que acontece ao acessar `/carrinho` ou `/checkout` diretamente pela URL sem estar logado?
- O que acontece ao logar como usuário comum e acessar `/admin` diretamente pela URL?
- O login não valida senha de verdade (qualquer senha com 3+ caracteres funciona). Isso é um problema de segurança do projeto, ou é uma simplificação esperada para fins didáticos? Justifique.

4.4 Persistência local (carrinho e sessão)

- O carrinho e o login são salvos no `localStorage`. O que acontece se esse dado for corrompido manualmente (veja o teste 9 na seção 5)?
- O carregamento do carrinho/sessão salvos está protegido com `try/catch`. O salvamento (toda vez que o carrinho muda) também está?

5. Casos de teste sugeridos

Execute cada teste abaixo manualmente, anote o resultado observado na ficha da seção 6 e guarde uma evidência (print de tela, print da aba Network/Application, ou trecho de código citado com nome do arquivo e linha).

Nº	O que testar	Como testar
1	Catálogo com API fora do ar	Em core/services/produtos.service.ts, troque a linha da constante API pela URL comentada logo abaixo dela (o endpoint "/products-erro", que não existe de propósito). Salve, recarregue /produtos e observe a tela. Depois desfaça a alteração.
2	Catálogo vazio	Ainda com a troca do teste 1, ou usando o DevTools para interceptar a resposta, verifique se existe alguma mensagem específica para "nenhum produto encontrado", ou se a tela fica apenas com o total zerado.
3	Rede totalmente offline	Em DevTools > Network, marque "Offline", clique em "Tentar novamente" no catálogo (ou recarregue /produtos) e observe se a tela sai do estado de carregamento com uma mensagem clara.
4	Acessar rota protegida sem login	Sem estar logado (ou após deslogar), digite a URL /carrinho ou /checkout diretamente na barra de endereços.
5	Acessar /admin como usuário comum	Faça login com um e-mail comum (ex.: aluno@teste.com) e, em seguida, digite /admin diretamente na URL.
6	Acessar /admin como administrador	Faça login com admin@email.com e acesse /admin normalmente pelo menu, confirmando que o acesso é liberado.
7	Token expirado / 401 simulado	Com o DevTools aberto na aba Application > Local Storage, apague a chave "minha-loja-token" (mas deixe "minha-loja-usuario") e tente navegar para /carrinho. Observe se o app trata essa inconsistência de forma razoável.
8	Rota inexistente	Digite uma URL que não existe, como /pagina-que-nao-existe, e veja para onde o app redireciona.
9	Dado corrompido no localStorage	Em DevTools > Application > Local Storage, edite manualmente o valor de "minha-loja-carrinho" para um texto inválido (ex.: "{quebrado}"). Recarregue a página e veja se o app trava, mostra erro no console, ou se recupera graciosamente.
10	Checkout com carrinho vazio	Estando logado e com o carrinho vazio, acesse /checkout diretamente e confirme a mensagem exibida.
11	Leitura de código do interceptor	Leia http.interceptor.ts por inteiro e explique, com suas palavras, o que acontece para cada status (401, 403, 500) e o que aconteceria com um status não tratado explicitamente (ex.: 404 ou 400).
12	Duplo clique em ações	No catálogo, clique duas vezes seguidas rapidamente em "Adicionar ao Carrinho" no mesmo produto e confira se o item foi adicionado uma ou duas vezes — é o comportamento esperado?

6. Ficha de registro de evidências

Preencha uma linha para cada teste executado (os 12 sugeridos, mais qualquer outro que você decidir investigar por conta própria). Copie a tabela para o seu relatório e preencha uma linha por teste.

Nº do teste	Passos executados	Resultado esperado	Resultado observado	Classificação

Classificação: use uma das três opções — “Falha real encontrada”, “Já tratado corretamente (comportamento esperado)” ou “Funciona, mas poderia ser melhor (sugestão de melhoria)”.

7. Entregável

Entregue um relatório curto (1 a 2 páginas), no formato de um relatório de revisão de código real, contendo:

- A ficha de registro da seção 6, preenchida com os 12 testes (mínimo) e as evidências.
- Uma conclusão geral: o tratamento de falhas da Minha Loja está adequado para os quatro pilares testados (API, interceptor, autenticação/autorização e persistência local)? Por quê?
- Se encontrou alguma falha real: descreva o cenário exato para reproduzi-la (passo a passo) e onde no código ela deveria ser corrigida.
- Se não encontrou nenhuma falha real: isso é uma conclusão válida. Descreva o que foi verificado e por que você confia nessa conclusão.

Lembre-se de reverter qualquer alteração feita no código (como a troca do endpoint no teste 1) antes de entregar o projeto — a auditoria não deve deixar o projeto quebrado para trás.

8. Critérios de avaliação

Critério	Peso	O que é avaliado
Cobertura da metodologia	30%	Os 12 testes sugeridos (ou mais) foram realmente executados e documentados, não só listados.
Qualidade da evidência	25%	Prints das abas Network/Application, trechos de código citados com arquivo e linha, passos reproduzíveis — não apenas “testei e funcionou”.
Leitura crítica do código	25%	Capacidade de explicar o funcionamento real do interceptor e dos guards, e de identificar (ou confirmar corretamente a ausência de) lacunas como estado vazio não tratado, logs de depuração esquecidos, ou proteção assimétrica de leitura/escrita no localStorage.
Comunicação do relatório	20%	Relatório claro, organizado e tecnicamente correto, sem jargão desnecessário.

Atenção: a nota não depende da quantidade de falhas encontradas. Um relatório rigoroso que não encontra problemas reais recebe a mesma nota máxima que um relatório rigoroso que encontra um problema real de verdade.

9. Reflexão final

Responda em poucas linhas, ao final do relatório:

- O interceptor HTTP trata erros de forma centralizada, enquanto lista-produtos.ts também trata erro localmente. Na sua opinião, essa divisão de responsabilidades é uma boa prática ou uma duplicação desnecessária? Justifique.
- Os guards de rota (auth e admin) protegem a navegação, mas o token e o perfil do usuário ficam salvos em localStorage, acessível a qualquer script no navegador. Isso seria seguro o bastante para uma loja de verdade? O que faltaria?
- Se você tivesse que auditar um sistema em produção sem nenhum tratamento de erro, por onde começaria?
- Cite uma melhoria (mesmo pequena, mesmo que não seja um “bug”) que você sugeriria para a Minha Loja depois desta auditoria.